Machine Coding: Tab Components (production level)

Problem: Build a composable Tabs component (`<Tabs>` , `<TabList>` , `<Tab>` , `<TabPanel>`) used in a real dashboard: the active tab lives in the URL, panels load their own data lazily, and it is fully keyboard and screen-reader accessible.
Tech stack: React 19 + JavaScript (Vite), plain CSS
Time limit: 60 minutes

PART A: Thought process

General steps for any machine coding task

1. Clarify what must work and what is out of scope. Write the assumptions down.
2. List the interactions: every click, key and input, and what it should do. This is the checklist.
3. Sketch the components and decide who owns which state.
4. Keep state as small as possible. Derive everything else.
5. Agree the contract (props / API shape) before writing JSX.
6. Find the hard parts early, so they get time instead of the last 5 minutes.
7. Build the simplest working version, then add one working layer at a time.
8. Cover states and edge cases: loading, error, empty, disabled, invalid input.
9. Wrap up: trade-offs and what you would add next.

Time budget (60 minutes)

Phase	Minutes
Requirements, questions, assumptions	5
Interactions, components, state, API contract	7
Compound components + context, click to switch	10
Accessibility + keyboard (roving tabindex, arrows, Home/End, manual mode)	10
URL sync (`?tab=` , Back/Forward, invalid values)	8
Lazy panels, data hook with cache, loading / error / retry	12
keepMounted, styling, manual test	5
Wrap-up and trade-offs	3

Think out loud

Say what you are doing and why before you type: "I'll put the tabs in context so `<Tab>` and `<TabPanel>` can be placed anywhere, and I'll read tab order from the DOM so I don't need a registration system." The

interviewer grades the reasoning as much as the code. When stuck, say what you are trying so they can nudge you.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Compound API: `<Tabs>` wraps a `<TabList>` of `<Tab value>` buttons and any number of `<TabPanel value>`; the consumer controls markup and order.
- Clicking a tab shows its panel; exactly one tab is active.
- Works **uncontrolled** (`defaultValue`) and **controlled** (`value` + `onChange`).
- Disabled tabs cannot be clicked or reached by keyboard.
- Keyboard: Left/Right (wrap), Home/End. `activation="automatic"` (focus selects) or `"manual"` (Enter/Space selects).
- Active tab is stored in the URL (`?tab=activity`): survives refresh, can be shared, Back/Forward work.
- Panels are **lazy**: a panel is not mounted until first opened. `keepMounted` keeps it (hidden) after that, so form drafts survive.
- Each data panel fetches its own data with loading, error + retry, and empty states; data is cached so returning to a tab is instant.

Non-functional requirements

Area	What it means here
Performance	Only visited panels mount; one request per tab (deduped and cached); context value memoized.
Scalability	Tabs read order from the DOM, so 3 or 30 tabs, conditional tabs and wrapped tabs need no extra code. The tab list scrolls horizontally on overflow.
Error handling	Failed fetch shows an error with Retry and is not cached; invalid or disabled `?tab=` falls back to the first tab; using `<Tab>` outside `<Tabs>` throws a clear error.
Accessibility	WAI-ARIA tabs pattern: `tablist` / `tab` / `tabpanel` , `aria-selected` , `aria-controls` , `aria-labelledby` , roving tabindex, visible focus ring.
Reusability	Tabs know nothing about URLs or fetching; URL sync and data are hooks the app composes.
Extensibility	Vertical orientation, closable tabs or an animated indicator can be added without changing the public API.

Clarifying questions

1. Do you want a compound API (`<Tab>` , `<TabPanel>` children) or a data-driven `items` prop?
2. Should arrow keys select a tab immediately, or only move focus (manual activation)?
3. Should the active tab be in the URL, and should every switch add a Back-button entry?
4. Should panel content load on first open or all upfront? Should it refetch when revisited?
5. Must a panel keep its local state (e.g. a half-filled form) when you switch away?
6. Is a routing library available, or plain browser APIs only?

Assumptions

- Compound components with string `value`s (unique and id-safe, e.g. `"overview"`).
- Automatic activation by default, manual as an option.
- URL via plain `history.pushState` + `popstate` (no router); each switch adds a history entry.
- Lazy by default; `keepMounted` opt-in. Data cached for the session; failed requests are not cached.
- Mock API with an 800 ms delay; the first Reports call fails so retry can be shown.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads with no `?tab=`	Overview is active; its panel shows "Loading…" then data
Page loads with `?tab=activity`	Activity is active on first render
Page loads with `?tab=billing` (disabled) or `?tab=xyz`	Falls back to Overview
Click a tab	Tab selected, panel shown, URL becomes `?tab=<value>`
Click a disabled tab	Nothing happens
Tab key into the tab list	Focus lands on the active tab only (one tab stop)
Right / Left arrow	Focus moves to next / previous enabled tab, wraps, skips disabled; selects it in automatic mode
Home / End	First / last enabled tab
Arrow in manual mode	Focus moves only; Enter or Space selects
Tab key from the tab list	Focus goes to the active panel
Browser Back / Forward	Active tab follows the URL
Open a tab for the first time	Panel mounts and fetches
Switch away mid-fetch, come back	No duplicate request; result shows when ready; no state update on an unmounted panel
Return to an already loaded tab	Data shows instantly from cache, no "Loading…" flash
Fetch fails	Error message + Retry; Retry refetches
Fetch returns `[]`	Empty message ("No reports yet.")
Type in Settings, switch, come back	Draft is still there (`keepMounted`)
Two `<Tabs>` on one page	No duplicate DOM ids
`<Tab>` used outside `<Tabs>`	Clear error thrown

3. Design the architecture

Module	Responsibility
`Tabs`	Owns (or receives) the active value; provides context: `active`, `select`, `activation`, `baseId`
`TabList`	`role="tablist"`; keyboard navigation; picks the first enabled tab when nothing is selected
`Tab`	One `role="tab"` button: selected state, roving tabindex, ARIA ids
`TabPanel`	`role="tabpanel"`; lazy mount and `keepMounted`
`nextTabIndex` (keyboard.js)	Pure function: key + current index -> next index (wrap)
`useQueryParam`	Syncs `?tab=` with state and Back/Forward
`useTabData`	Per-tab fetch with module-level cache, dedupe, stale guard, retry
`DataPanel`	Renders loading / error / empty / success for one tab's data
`api.js`	Mock endpoint
`App`	Composes it: list of tabs, URL value validation, panels

```
App
 |- useQueryParam('tab')           (URL <-> state)
 |- <Tabs value onChange>          (context provider)
 |    |- <TabList>                 (keyboard, fallback)
 |    |    |- <Tab value> x N
 |    |- <TabPanel value>          (lazy / keepMounted)
 |         |- <DataPanel tabKey>   (useTabData -> api.fetchTab)
 |- <Tabs defaultValue activation="manual">  (second, uncontrolled instance)
```

Start simple: one `Tabs.jsx` with all four parts. URL and data are separate hooks because the Tabs component must not know about either.

4. Design the state / data model

State or field	Owner	Purpose
`innerValue`	`Tabs` (uncontrolled only)	Active tab when the parent does not control it
`value` prop	Parent (controlled)	Active tab from outside, here from the URL
`tabParam`	`useQueryParam`	Raw `?tab=` value, updated on click and on `popstate`
`visited`	each `TabPanel`	Has this panel ever been opened (for `keepMounted`)
`{ status, data, error }`	`useTabData`	Fetch state for one panel
`attempt`	`useTabData`	Bumped by Retry to re-run the effect

State or field	Owner	Purpose
`cache` (Map)	module scope	`key -> { promise, data }`, outlives unmounted panels

```
// Tabs
const [innerValue, setInnerValue] = useState(defaultValue);
const isControlled = value !== undefined;
const active = isControlled ? value : innerValue;

// useTabData
const [state, setState] = useState(() => initialState(key)); // success from cache, else loading
const [attempt, setAttempt] = useState(0);
```

Senior note: derive, don't store.

- `selected` for each tab and panel is `active === value`, never stored.
- The valid active tab in `App` is derived during render: `isSelectable(tabParam) ? tabParam : TABS[0].id`. No effect that "fixes" state, so no extra render and no flicker.
- The list of tabs and their order is not stored in React at all: the DOM already has it.
- `isControlled` is derived from `value !== undefined`.

5. Design the data structures and contracts

Example data

```
const TABS = [
  { id: 'overview', label: 'Overview' },
  { id: 'activity', label: 'Activity' },
  { id: 'reports', label: 'Reports' },
  { id: 'settings', label: 'Settings' },
  { id: 'billing', label: 'Billing', disabled: true },
];
```

Stable string ids, not indexes: they go in the URL, in DOM ids and in the cache key, and they don't change when a tab is inserted or hidden.

Component contracts

Component	Props
`Tabs`	`value?`, `defaultValue?`, `onChange?(value)`, `activation?: 'automatic' | 'manual'`, `children`
`TabList`	`label` (accessible name), `children`
`Tab`	`value`, `disabled?`, `children`
`TabPanel`	`value`, `keepMounted?`, `children`

```
<Tabs value={activeTab} onChange={setTabParam}>
  <TabList label="Account sections">
    <Tab value="overview">Overview</Tab>
    <Tab value="billing" disabled>Billing</Tab>
  </TabList>
  <TabPanel value="overview">...</TabPanel>
</Tabs>
```

Hook contracts

```
const [value, setValue] = useQueryParam('tab');      // string | null, (next) => void
const { status, data, error, retry } = useTabData('reports', fetchTab);
// status: 'loading' | 'success' | 'error'
```

Mock API

```
fetchTab('overview') // -> Promise<{ users: 1284, revenue: '$48.2k', churn: '2.1%' }>
fetchTab('activity') // -> Promise<[{ id: 'a1', text: 'Asha upgraded...', time: '2m ago' }, ...]>
fetchTab('reports')  // -> first call rejects Error('Could not load reports (500).'), then []
```

6. Trace the data flow

User presses Right arrow on "Overview" (automatic mode), Reports never opened before:

```
keydown 'ArrowRight' on Overview
  -> TabList.handleKeyDown: enabled tabs read from DOM, nextTabIndex -> 1
  -> tab.focus(), scrollIntoView, select('activity')
  -> Tabs.select: controlled, so only onChange('activity')
  -> useQueryParam.update: history.pushState(?tab=activity), setValue('activity')
  -> App re-renders: activeTab = 'activity' (valid)
  -> context value changes -> Tab re-renders: aria-selected + tabIndex=0 move to Activity
  -> TabPanel 'overview' returns null (unmounts), TabPanel 'activity' mounts
  -> DataPanel -> useTabData('activity'): cache miss -> status 'loading' -> "Loading…"
  -> effect: load() stores { promise } in cache, fetchTab('activity')
  -> 800 ms later: entry.data = items, setState({ status: 'success' })
  -> list renders
Later, Back button:
  -> popstate -> setValue(read('tab')) -> App re-renders with 'overview'
  -> Overview panel mounts, initialState finds cached data -> renders instantly
```

7. Important technical challenges

Challenge 1: Compound components without a registration system

Problem: With `<Tab>` children, `Tabs` does not receive an array, so it cannot know the order of tabs, which are disabled, or what "next tab" means. A common answer is to make each `<Tab>` register itself into an array in context, but that is fragile: registration order breaks with conditional tabs, StrictMode double effects and re-ordering.

Solution: Context carries only `active` and `select`. Each `Tab` decides `selected` itself. For keyboard order, read the enabled tabs from the DOM at keydown time: DOM order is visual order, and `:disabled` filters disabled ones for free.

```
const TAB_SELECTOR = '[role="tab"]:not(:disabled)';

function handleKeyDown(e) {
  const tabs = [...listRef.current.querySelectorAll(TAB_SELECTOR)];
  const index = nextTabIndex(e.key, tabs.indexOf(document.activeElement), tabs.length);
  if (index === null) return;
  e.preventDefault();
  tabs[index].focus();
  if (activation === 'automatic') select(tabs[index].dataset.value);
}
```

Challenge 2: Accessibility, roving tabindex and manual activation

Problem: With 5 tabs, a keyboard user should not press Tab 5 times to reach the content. Screen readers need to know which tab is selected and which panel it controls. Two `<Tabs>` on one page must not produce duplicate ids.

Solution: Only the selected tab has `tabIndex=0`, all others `-1` (one tab stop; arrows move inside). `useId()` gives a unique prefix per `<Tabs>` instance. In manual mode arrows only move focus; the native `<button>` already turns Enter/Space into a click, so no extra key code is needed.

```
<button
  role="tab"
  id={`${baseId}-tab-${value}`}
  aria-controls={`${baseId}-panel-${value}`}
  aria-selected={selected}
  tabIndex={selected ? 0 : -1}
  disabled={disabled}
  onClick={() => select(value)}
/>
```

```
export function nextTabIndex(key, current, count) {
  const target = { ArrowRight: current + 1, ArrowLeft: current - 1, Home: 0, End: count - 1 }[key];
  if (target === undefined || count === 0) return null;
  return (target + count) % count; // -1 -> last, count -> 0
}
```

Challenge 3: Keeping the active tab in the URL

Problem: Refresh resets to the first tab; links can't point at a tab; Back does nothing. And the URL is user input: `?tab=billing` (disabled) or `?tab=xyz` must not leave the UI with no selected tab.

Solution: A small `useQueryParam` hook: `pushState` on change, `popstate` listener for Back/Forward. The app validates during render and passes a safe value to controlled `<Tabs>`.

```
const update = (next) => {
  const url = new URL(window.location.href);
  url.searchParams.set(name, next);
  window.history.pushState(null, '', url);
  setValue(next);
};
useEffect(() => {
  const onPopState = () => setValue(read(name));
  window.addEventListener('popstate', onPopState);
  return () => window.removeEventListener('popstate', onPopState);
}, [name]);

// App
const activeTab = isSelectable(tabParam) ? tabParam : TABS[0].id;
```

Challenge 4: Lazy panels that fetch, with cache, dedupe and race safety

Problem: Scenario: user opens Activity (fetch starts), switches to Overview after 200 ms, then back to Activity. Naive code: (1) fires a second request, (2) calls `setState` on an unmounted panel, (3) shows "Loading…" again even after data arrived. A failed request must not be cached forever.

Solution: A module-level cache keyed by tab storing the **promise** (dedupes in-flight requests) and the resolved **data** (instant render on remount, no flash). An `ignore` flag in the effect cleanup drops results for an unmounted panel. Rejected promises delete their cache entry so Retry fetches again.

```
const cache = new Map(); // key -> { promise, data }

function load(key, fetcher) {
  let entry = cache.get(key);
  if (!entry) {
    entry = { promise: fetcher(key), data: undefined };
    entry.promise.then((data) => (entry.data = data), () => cache.delete(key));
    cache.set(key, entry);
  }
  return entry;
}

useEffect(() => {
  let ignore = false;
  load(key, fetcher).promise.then(
```

```
    (data) => !ignore && setState({ status: 'success', data, error: null }),
    (error) => !ignore && setState({ status: 'error', data: null, error }),
  );
  return () => { ignore = true; };
}, [key, fetcher, attempt]);
```

`keepMounted` uses the "adjust state during render" pattern so there is no extra effect:

```
const [visited, setVisited] = useState(selected);
if (selected && !visited) setVisited(true);
if (!selected && !(keepMounted && visited)) return null;
```

8. Implement in the correct order

#	Step	Working result
1	Vite app, hard-coded buttons + `useState` active id	Click switches content
2	Split into `Tabs` / `TabList` / `Tab` / `TabPanel` with context	Same behavior, compound API
3	`useTabs()` guard, `useId` ids, ARIA roles and attributes	Screen-reader correct
4	Roving tabindex + `nextTabIndex` + keyboard handler	Arrows, Home/End, wrap, skip disabled
5	`activation="manual"`	Arrows move focus only
6	Controlled vs uncontrolled (`value` / `defaultValue` / `onChange`)	Parent can drive it
7	First-enabled fallback in `TabList`	No `defaultValue` still works
8	`useQueryParam` + render-time validation in App	Refresh, share, Back/Forward
9	Lazy `TabPanel` + `keepMounted`	Unvisited panels never mount; drafts survive
10	Mock API + `useTabData` (loading / success)	Panels load their data
11	Cache, dedupe, ignore flag, error + Retry, empty	All data states
12	CSS: focus ring, disabled, horizontal scroll; manual test of the use-case table	Done

9. Cover all states and edge cases

State / edge case	Handling
Initial (no URL param)	First tab active, its data loading
Loading	"Loading…" with `role="status"` (announced)
Success	Panel content
Empty (`[]`)	"No reports yet."

State / edge case	Handling
Error	Message with `role="alert"` + Retry; failed request not cached
Retry	Back to loading, new request
Cached revisit	Instant, no loading flash, no request
Switch away mid-fetch	Request continues once, result cached, no setState on unmounted panel
Disabled tab	Native `disabled`: no click, no focus, skipped by arrows
`?tab=` disabled or unknown	Falls back to first tab during render
No `defaultValue` (uncontrolled)	`TabList` selects first enabled tab before paint (`useLayoutEffect`)
All tabs disabled	Nothing selected, nothing crashes
Many tabs	List scrolls horizontally; focused tab scrolled into view
Two instances on a page	`useId` prefix keeps ids unique
`<Tab>` outside `<Tabs>`	Throws "must be used inside <Tabs>"
StrictMode double effect	Same cached promise reused, still one request

10. Final structure

```
_48_tab_components/
  index.html
  package.json
  vite.config.js
  src/
    main.jsx
    index.css
    App.jsx
    components/
      DataPanel.jsx
      Tabs/
        Tabs.jsx          Tabs, TabList, Tab, TabPanel
        TabsContext.js    context + useTabs guard
        keyboard.js       nextTabIndex (pure)
        Tabs.css
    hooks/
      useQueryParam.js
      useTabData.js
    services/
      api.js
```

11. Full working code

src/components/Tabs/TabsContext.js

```
import { createContext, useContext } from 'react';

export const TabsContext = createContext(null);

export function useTabs() {
  const ctx = useContext(TabsContext);
  if (!ctx) throw new Error('Tab, TabList and TabPanel must be used inside <Tabs>');
  return ctx;
}
```

src/components/Tabs/keyboard.js

```
// Returns the index to move focus to, or null if the key is not a tab-navigation key.
export function nextTabIndex(key, current, count) {
  const target = { ArrowRight: current + 1, ArrowLeft: current - 1, Home: 0, End: count - 1 }[key];
  if (target === undefined || count === 0) return null;
  return (target + count) % count; // wraps -1 -> last and count -> 0
}
```

src/components/Tabs/Tabs.jsx

```
import { useCallback, useId, useLayoutEffect, useMemo, useRef, useState } from 'react';
import { TabsContext, useTabs } from './TabsContext';
import { nextTabIndex } from './keyboard';
import './Tabs.css';

const TAB_SELECTOR = '[role="tab"]:not(:disabled)';

export function Tabs({ value, defaultValue, onChange, activation = 'automatic', children }) {
  const [innerValue, setInnerValue] = useState(defaultValue);
  const isControlled = value !== undefined;
  const active = isControlled ? value : innerValue;
  const baseId = useId();

  const select = useCallback(
    (next) => {
      if (next === active) return;
      if (!isControlled) setInnerValue(next);
      onChange?.(next);
    },
    [active, isControlled, onChange],
  );

  const ctx = useMemo(() => ({ active, select, activation, baseId }), [active, select, activation,
```

```
baseId]);

  return (
    <TabsContext.Provider value={ctx}>
      <div className="tabs">{children}</div>
    </TabsContext.Provider>
  );
}

export function TabList({ label, children }) {
  const { active, select, activation } = useTabs();
  const listRef = useRef(null);

  // No tab selected (no defaultValue, or it pointed at a missing tab): pick the first enabled one.
  useLayoutEffect(() => {
    const list = listRef.current;
    if (list.querySelector('[role="tab"][aria-selected="true"]')) return;
    const first = list.querySelector(TAB_SELECTOR);
    if (first) select(first.dataset.value);
  }, [active, select]);

  function handleKeyDown(e) {
    // DOM order is the visual order, so read enabled tabs from the DOM instead of registering them.
    const tabs = [...listRef.current.querySelectorAll(TAB_SELECTOR)];
    const index = nextTabIndex(e.key, tabs.indexOf(document.activeElement), tabs.length);
    if (index === null) return;
    e.preventDefault();
    const tab = tabs[index];
    tab.focus();
    tab.scrollIntoView({ block: 'nearest', inline: 'nearest' });
    if (activation === 'automatic') select(tab.dataset.value);
  }

  return (
    <div ref={listRef} role="tablist" aria-label={label} className="tab-list" onKeyDown=
{handleKeyDown}>
      {children}
    </div>
  );
}

export function Tab({ value, disabled = false, children }) {
  const { active, select, baseId } = useTabs();
  const selected = active === value;

  return (
    <button
```

```
      type="button"
      role="tab"
      id={`${baseId}-tab-${value}`}
      aria-controls={`${baseId}-panel-${value}`}
      aria-selected={selected}
      tabIndex={selected ? 0 : -1}
      disabled={disabled}
      data-value={value}
      className="tab"
      onClick={() => select(value)}
    >
      {children}
    </button>
  );
}

export function TabPanel({ value, keepMounted = false, children }) {
  const { active, baseId } = useTabs();
  const selected = active === value;
  const [visited, setVisited] = useState(selected);
  if (selected && !visited) setVisited(true); // adjust state during render, no extra effect

  // Lazy: never mounted until first opened. keepMounted: stays mounted (hidden) after that.
  if (!selected && !(keepMounted && visited)) return null;

  return (
    <div
      role="tabpanel"
      id={`${baseId}-panel-${value}`}
      aria-labelledby={`${baseId}-tab-${value}`}
      hidden={!selected}
      tabIndex={0}
      className="tab-panel"
    >
      {children}
    </div>
  );
}
```

src/components/Tabs/Tabs.css

```
.tabs {
  border: 1px solid #e2e8f0;
  border-radius: 10px;
  background: #fff;
}
```

```css
.tab-list {
  display: flex;
  gap: 4px;
  padding: 0 8px;
  border-bottom: 1px solid #e2e8f0;
  overflow-x: auto;
  scrollbar-width: thin;
}

.tab {
  flex-shrink: 0;
  padding: 12px 16px;
  border: 0;
  border-bottom: 2px solid transparent;
  background: none;
  color: #475569;
  font: inherit;
  font-weight: 500;
  cursor: pointer;
}

.tab:hover:not(:disabled) {
  color: #0f172a;
}

.tab[aria-selected='true'] {
  color: #4f46e5;
  border-bottom-color: #4f46e5;
}

.tab:focus-visible,
.tab-panel:focus-visible {
  outline: 2px solid #4f46e5;
  outline-offset: -2px;
}

.tab:disabled {
  color: #cbd5e1;
  cursor: not-allowed;
}

.tab-panel {
  padding: 20px;
}
```

src/hooks/useQueryParam.js

```
import { useCallback, useEffect, useState } from 'react';

const read = (name) => new URLSearchParams(window.location.search).get(name);

// Keeps one query param (e.g. ?tab=activity) in sync with React state and the Back/Forward buttons.
export function useQueryParam(name) {
  const [value, setValue] = useState(() => read(name));

  useEffect(() => {
    const onPopState = () => setValue(read(name));
    window.addEventListener('popstate', onPopState);
    return () => window.removeEventListener('popstate', onPopState);
  }, [name]);

  const update = useCallback(
    (next) => {
      const url = new URL(window.location.href);
      url.searchParams.set(name, next);
      window.history.pushState(null, '', url);
      setValue(next);
    },
    [name],
  );

  return [value, update];
}
```

src/hooks/useTabData.js

```
import { useEffect, useState } from 'react';

// key -> { promise, data }. Module level, so it outlives panels that unmount on tab switch.
const cache = new Map();

function load(key, fetcher) {
  let entry = cache.get(key);
  if (!entry) {
    entry = { promise: fetcher(key), data: undefined };
    entry.promise.then(
      (data) => (entry.data = data),
      () => cache.delete(key), // failed requests are not cached, so Retry fetches again
    );
    cache.set(key, entry);
  }
  return entry;
```

```
}

const LOADING = { status: 'loading', data: null, error: null };

function initialState(key) {
  const data = cache.get(key)?.data;
  return data === undefined ? LOADING : { status: 'success', data, error: null };
}

// key must stay the same for the life of the component (one key per panel).
export function useTabData(key, fetcher) {
  const [state, setState] = useState(() => initialState(key));
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    let ignore = false; // panel unmounted (user switched tab) before the request finished
    load(key, fetcher).promise.then(
      (data) => !ignore && setState({ status: 'success', data, error: null }),
      (error) => !ignore && setState({ status: 'error', data: null, error }),
    );
    return () => {
      ignore = true;
    };
  }, [key, fetcher, attempt]);

  const retry = () => {
    setState(LOADING);
    setAttempt((a) => a + 1);
  };

  return { ...state, retry };
}
```

src/services/api.js

```
// Mock API: each tab's content comes from its own "endpoint".
const DATA = {
  overview: { users: 1284, revenue: '$48.2k', churn: '2.1%' },
  activity: [
    { id: 'a1', text: 'Asha upgraded to the Pro plan', time: '2m ago' },
    { id: 'a2', text: 'Ravi invited 3 teammates', time: '1h ago' },
    { id: 'a3', text: 'Meera exported the Q3 report', time: 'Yesterday' },
  ],
  reports: [],
};

let reportCalls = 0;
```

```
export function fetchTab(key) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      // The first reports call fails so the error + retry path can be seen.
      if (key === 'reports' && ++reportCalls === 1) reject(new Error('Could not load reports
(500).'));
      else resolve(DATA[key]);
    }, 800);
  });
}
```

src/components/DataPanel.jsx

```
import { useTabData } from '../hooks/useTabData';
import { fetchTab } from '../services/api';

export function DataPanel({ tabKey, emptyText, children }) {
  const { status, data, error, retry } = useTabData(tabKey, fetchTab);

  if (status === 'loading') return <p className="muted" role="status">Loading…</p>;

  if (status === 'error') {
    return (
      <div className="error" role="alert">
        <span>{error.message}</span>
        <button type="button" onClick={retry}>Retry</button>
      </div>
    );
  }

  if (Array.isArray(data) && data.length === 0) return <p className="muted">{emptyText}</p>;

  return children(data);
}
```

src/App.jsx

```
import { Tab, TabList, TabPanel, Tabs } from './components/Tabs/Tabs';
import { DataPanel } from './components/DataPanel';
import { useQueryParam } from './hooks/useQueryParam';

const TABS = [
  { id: 'overview', label: 'Overview' },
  { id: 'activity', label: 'Activity' },
  { id: 'reports', label: 'Reports' },
```

```
  { id: 'settings', label: 'Settings' },
  { id: 'billing', label: 'Billing', disabled: true },
];

const isSelectable = (id) => TABS.some((t) => t.id === id && !t.disabled);

export default function App() {
  const [tabParam, setTabParam] = useQueryParam('tab');
  // Bad or disabled ?tab= values fall back to the first tab, derived during render.
  const activeTab = isSelectable(tabParam) ? tabParam : TABS[0].id;

  return (
    <main className="page">
      <h1>Account dashboard</h1>

      <Tabs value={activeTab} onChange={setTabParam}>
        <TabList label="Account sections">
          {TABS.map((t) => (
            <Tab key={t.id} value={t.id} disabled={t.disabled}>
              {t.label}
            </Tab>
          ))}
        </TabList>

        <TabPanel value="overview">
          <DataPanel tabKey="overview">
            {(d) => (
              <dl className="stats">
                <div><dt>Users</dt><dd>{d.users}</dd></div>
                <div><dt>Revenue</dt><dd>{d.revenue}</dd></div>
                <div><dt>Churn</dt><dd>{d.churn}</dd></div>
              </dl>
            )}
          </DataPanel>
        </TabPanel>

        <TabPanel value="activity">
          <DataPanel tabKey="activity" emptyText="No activity yet.">
            {(items) => (
              <ul className="list">
                {items.map((a) => (
                  <li key={a.id}>{a.text} <span className="muted">{a.time}</span></li>
                ))}
              </ul>
            )}
          </DataPanel>
        </TabPanel>
```

```
        <TabPanel value="reports">
          <DataPanel tabKey="reports" emptyText="No reports yet.">
            {(items) => <ul className="list">{items.map((r) => <li key={r.id}>{r.name}</li>)}</ul>}
          </DataPanel>
        </TabPanel>

        {/* keepMounted: the unsaved draft survives switching tabs */}
        <TabPanel value="settings" keepMounted>
          <label className="field">
            Display name
            <input placeholder="Type, switch tab, come back" />
          </label>
        </TabPanel>
      </Tabs>

      <h2>Manual activation (uncontrolled)</h2>
      <Tabs defaultValue="html" activation="manual">
        <TabList label="Code samples">
          <Tab value="html">HTML</Tab>
          <Tab value="css">CSS</Tab>
          <Tab value="js">JavaScript</Tab>
        </TabList>
        <TabPanel value="html">Arrow keys move focus; Enter or Space opens the tab.</TabPanel>
        <TabPanel value="css">CSS panel</TabPanel>
        <TabPanel value="js">JavaScript panel</TabPanel>
      </Tabs>
    </main>
  );
}
```

src/index.css

```
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, -apple-system, 'Segoe UI', Roboto, sans-serif;
  color: #0f172a;
  background: #f8fafc;
}

.page {
  max-width: 720px;
  margin: 0 auto;
```

```
  padding: 32px 16px;
}

h2 {
  margin-top: 40px;
  font-size: 1.1rem;
}

.muted {
  color: #64748b;
}

.stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(140px, 1fr));
  gap: 12px;
  margin: 0;
}

.stats div {
  padding: 12px;
  border-radius: 8px;
  background: #f1f5f9;
}

.stats dt {
  font-size: 0.85rem;
  color: #64748b;
}

.stats dd {
  margin: 4px 0 0;
  font-size: 1.4rem;
  font-weight: 600;
}

.list {
  margin: 0;
  padding-left: 18px;
  line-height: 1.9;
}

.error {
  display: flex;
  align-items: center;
  gap: 12px;
  color: #b91c1c;
```

```
}

.error button {
  padding: 6px 12px;
  border: 1px solid #b91c1c;
  border-radius: 6px;
  background: #fff;
  color: #b91c1c;
  cursor: pointer;
}

.field {
  display: grid;
  gap: 6px;
  max-width: 320px;
}

.field input {
  padding: 8px 10px;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font: inherit;
}
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

12. Wrap-up

Edge cases handled

Disabled tabs (click, focus, arrows), wrap-around, invalid or disabled URL values, no default value, two instances on one page, overflowing tab list, in-flight request dedupe, unmounted-panel updates, cached revisits with no flash, failed requests not cached, empty data, drafts kept with `keepMounted`, StrictMode double effects.

Trade-offs to mention

- **DOM query vs registration:** reading tabs from the DOM is simple and always in visual order, but it couples keyboard logic to the markup (a `Tab` must render a `role="tab"` button). Registration works without the DOM but is far more code.
- **Lazy by default:** faster first render and fewer requests, but switching to a new tab shows a loader. Prefetch on hover would hide it.
- **pushState per switch:** Back walks through tabs, which some users expect and others find noisy, especially with arrow keys in automatic mode. `replaceState` is the one-line alternative.
- **Module-level cache:** simple and survives unmounts, but never expires and is global. In production use React Query / SWR (stale time, invalidation, background refetch).
- **Panels with `keepMounted` stay in the DOM:** keeps state, costs memory. Fine for a form, not for 20 heavy charts.

With more time

- Tests (React Testing Library + user-event): arrows, Home/End, disabled skip, manual mode, URL fallback, retry. `nextTabIndex` unit tests.
- Vertical orientation (`aria-orientation` , Up/Down keys).
- Animated active indicator (measure the selected tab with `ResizeObserver`).
- Prefetch a tab's data on hover / focus.
- `AbortController` to cancel requests nobody will use.
- Cache expiry / stale-while-revalidate, or move to React Query.
- Closable / draggable tabs, overflow "more" menu.

13. Counter questions the interviewer may ask

Requirements

Q1. Why a compound API instead of an `items` array prop?

Compound components let the consumer control markup: icons, badges, tabs and panels in different places, conditional tabs, custom wrappers. An `items` prop is simpler but every new need becomes a new prop. For a design-system component, compound wins; for a one-off, `items` is fine.

Q2. Automatic or manual activation: which is the default and why?

Automatic is the default because switching is cheap (cached, lazy panels) and it is what most users expect. Manual is better when selecting a tab is expensive (a heavy fetch per tab), so moving focus through tabs does not fire requests.

Q3. What if the product does not want Back to step through tabs?

Swap `pushState` for `replaceState` in `useQueryParam` (or add a `replace` option). The URL still survives refresh and sharing; history stays clean.

State design

Q4. Why is the URL value validated in App and not inside `<Tabs>`?

App knows the tab list and which tabs are disabled; `<Tabs>` intentionally does not. Validating during render means no effect, no extra render and no flash of a wrong tab. `<Tabs>` still has a minimal safety net: if nothing is selected, `TabList` picks the first enabled tab.

Q5. Why `useLayoutEffect` for that fallback and not `useEffect`?

It runs before the browser paints, so the user never sees a frame with no selected tab. A `useEffect` would paint once with nothing selected, then fix it.

Q6. What happens if a component switches from uncontrolled to controlled?

`isControlled` is derived each render, so it would start following `value`, but `innerValue` is stale if it switches back. Like native inputs, this is a misuse; in dev you can warn when `isControlled` changes between renders (a ref holding the first value).

Q7. Why memoize the context value?

Without `useMemo` every render of `Tabs` creates a new object, and every `Tab`, `TabList` and `TabPanel` consumer re-renders even when nothing changed. With it, consumers re-render only when `active`, `select`, `activation` or `baseId` change.

Challenge 1: compound components

Q8. Why not let each `<Tab>` register itself in an array?

Registration needs effects, ordering logic and cleanup; conditional tabs and StrictMode double-invoked effects can scramble the order. The DOM already knows the order and which buttons are disabled, so the keydown handler reads it once, at the moment it is needed.

Q9. Isn't touching the DOM "un-React"?

Focus is DOM behavior; React has no declarative focus API. We read the DOM only inside an event handler, never during render, which is the accepted place for imperative work.

Challenge 2: accessibility

Q10. What is roving tabindex and why use it?

Only one element in a composite widget is in the tab order (`tabIndex=0`); the rest are `-1` and reached with arrows. The whole tab list is one Tab stop, and the next Tab press goes to the panel. This is what the WAI-ARIA tabs pattern requires.

Q11. `aria-controls` points to a panel id that does not exist when the panel is lazy. Is that a problem?

The selected tab always has its panel in the DOM, which is what assistive tech uses. For unselected tabs the reference is dangling; most screen readers ignore it. If an audit tool complains, render an empty hidden panel for every tab and lazy-mount only its children.

Q12. Why native `disabled` instead of `aria-disabled`?

Native `disabled` blocks clicks, removes the button from focus and matches `:disabled` in the keyboard selector, all for free. `aria-disabled` keeps it focusable (useful if you want a tooltip explaining why), but then you must block activation yourself.

Q13. Why `useId` and not `tab-${value}`?

Two `<Tabs>` with a tab called `overview` would produce duplicate ids and break `aria-labelledby`. `useId` is unique per instance and stable between server and client render.

Challenge 3: URL sync

Q14. Why not read `window.location` directly during render?

React would not re-render when the URL changes (Back/Forward), and reading globals during render breaks SSR. The hook keeps the value in state and updates it on `popstate`.

Q15. What if the app already uses React Router?

Use `useSearchParams()` from the router instead of this hook; `<Tabs>` does not change because it only needs `value` and `onChange`. That separation is exactly why URL logic is not inside the component.

Challenge 4: data loading

Q16. Why cache the promise and not just the data?

If the user leaves and returns while the request is in flight, a data-only cache misses and fires a second request. Caching the promise dedupes in-flight calls; caching the data as well gives an instant render with no loading flash.

Q17. Why the `ignore` flag? Why not `AbortController`?

`ignore` stops a finished request from updating an unmounted panel or an outdated effect. We do not abort because the result is still useful: it lands in the cache for when the user returns. `AbortController` is the right choice when the result will never be used (e.g. search-as-you-type).

Q18. Why does the cache delete failed requests?

Otherwise the rejected promise would be cached forever and Retry would instantly fail again. Deleting it means the next `load()` call creates a fresh request.

Q19. What if a panel's data is 10,000 rows?

Fetching is the same; rendering is the problem. Paginate on the server or virtualize the list inside the panel. Do not use `keepMounted` for that panel, so its DOM is freed when hidden.

Q20. What if the API is slow (5 seconds)?

Show a skeleton instead of text, prefetch on hover/focus of the tab, and consider manual activation so arrowing past a tab does not start requests. Cached tabs stay instant.

Performance and scale

Q21. What if there are 50 tabs?

Keyboard and ARIA logic is unchanged. Visually, the list already scrolls horizontally and `scrollIntoView` keeps the focused tab visible; for a nicer UX add scroll buttons or an overflow "More" menu.

Q22. Does every tab re-render when the active tab changes?

Yes, all consumers of the context re-render, but a `Tab` is one button, so this is negligible. If panels were expensive they are already unmounted (lazy) or hidden; you could wrap panel children in `memo` if profiling showed a cost.

Trade-offs and testing

Q23. Why not use Radix / Headless UI?

In production, probably yes: they handle edge cases (RTL, vertical, focus restoration) and are well tested. In an interview the point is to show you understand the pattern, which is also what you need to debug or extend a library.

Q24. Why `hidden` for keepMounted panels instead of a CSS class?

`hidden` hides the panel from sight and from the accessibility tree in one attribute, with no CSS dependency. A class with `display: none` works too, but `visibility` or `opacity` tricks would leave the panel readable by screen readers.

Q25. How would you test this?

Unit-test `nextTabIndex` (wrap, Home/End, unknown key, empty list). With React Testing Library + user-event: click and arrow keys update `aria-selected`, disabled tabs are skipped, manual mode needs Enter, `?tab=xyz` falls back, mocked fetch shows loading -> error -> retry -> data, and the draft survives switching in `keepMounted`.